

Claude Chat:

> go through the entire code and find potential error and fixes, do not implement anything yet

Ran 2 shell commands (ctrl+o to expand)

- I read all four source files and the README. The game runs as written. Below is everything I found, most important first. I haven't changed any code.

1. Main bug: the player can walk off the screen (README Task 1)

Where: player.py:15-22 (game/player.py#L15-L22)

move_left() and move_right() change self.x with no limit, and update() does nothing. The player can walk past either edge and survive forever. Anvils only spawn at x from 20 to 640, so they can't reach a player who is off-screen.

Fix: clamp x in Player.update(), which already runs every frame after movement (game_engine.py:39 (game/game_engine.py#L39)):

```
def update(self):  
    self.x = max(0, min(self.x, self.screen_width - self.width))
```

2. Gameplay and logic issues

#: 2

Where: anvil.py:17-18 (game/anvil.py#L17-L18), game_engine.py:55 (game/game_engine.py#L55)

Problem: Anvils are only removed at screen_height + 10, but the ground line is

at height - 20. They visibly sink through the floor. This also gets in the

way of Task 4, which needs a "hit the ground" moment.

Suggested fix: Add an `has_hit_ground(ground_y)` check (`self.y + self.height >= ground_y`). Remove the anvil there, and later trigger the impact effects from the same point.

#: 3

Where: `game_engine.py:49-56` (`game/game_engine.py#L49-L56`)

Problem: After a collision sets `GAME_OVER`, the loop keeps moving and removing the other anvils for the rest of that frame.

Suggested fix: `break` (or `return`) right after setting `GAME_OVER`.

#: 4

Where: `player.py:8-9` (`game/player.py#L8-L9`) vs `player.py:29-34`

(`game/player.py#L29-L34`)

Problem: The hitbox is 44 px wide, but the drawn torso is only 32 px (`x+6` to `x+38`). The player gets hit when an anvil clearly misses. Meanwhile the helmet is drawn at `y - 2`, slightly above the hitbox.

Suggested fix: Shrink the collision rect to match the sprite (e.g. `rect.inflate(-12, 0)`), or move the helmet inside the box.

#: 5

Where: `anvil.py:21-22` (`game/anvil.py#L21-L22`)

Problem: The anvil's hitbox is the full 40×32 box, but its top part is only 32

px wide. The top corners are invisible "hit" pixels.

Suggested fix: Minor; use a slightly smaller rect or two rects.

#: 6

Where: `game_engine.py:41-46` (`game/game_engine.py#L41-L46`) vs `anvil.py:15`

(`game/anvil.py#L15`), `player.py:16-19` (`game/player.py#L16-L19`)

Problem: Spawning and the timer use real time (`get_ticks`), but movement is a

fixed number of pixels per frame. If the frame rate drops, anvils slow down while spawns and the score keep going. The same happens while the window is being dragged on Windows (the game loop freezes but `get_ticks` keeps counting).

Suggested fix: Use `dt = clock.tick(FPS) / 1000`, pass it to `update(dt)`, and scale speeds by it. Count survival time from `dt` too.

#: 7

Where: `anvil.py:10` (`game/anvil.py#L10`)

Problem: `random.randint(20, screen_width - 60)` raises `ValueError` if `screen_width < 80`. Not a problem at 700 px.

Suggested fix: Guard it, or derive the margins from the width.

3. Code quality

#: 8

Where: `game_engine.py:84-86` (`game/game_engine.py#L84-L86`)

Problem: A full-screen alpha Surface is created every frame during game over.

Suggested fix: Create it once in `__init__`.

#: 9

Where: `game_engine.py:18` (`game/game_engine.py#L18`)

Problem: Game states are raw strings ("`PLAYING`", "`GAME_OVER`"), so a typo fails silently.

Suggested fix: Use module constants or an Enum.

#: 10

Where: `game_engine.py:69` (`game/game_engine.py#L69`), `player.py:13` (`game/player.py#L13`)

Problem: The ground height 20 is hard-coded separately in both files.

Suggested fix: Share one GROUND_HEIGHT constant.

#: 11

Where: game_engine.py:13 (game/game_engine.py#L13)

Problem: spawn_delay is fixed and anvil speed is a fixed range, so there's no difficulty scaling.

Suggested fix: This is README Task 2: lower spawn_delay toward a minimum and raise the speed range as survival_time grows.

#: 12

Where: game_engine.py:43 (game/game_engine.py#L43)

Problem: get_ticks() is called twice per frame (lines 41 and 43).

Suggested fix: Call it once.

4. Project and repo

#: 13

Where: README.md:49-51 (README.md#L49-L51)

Problem: Task 2's description is a copy of Task 1's text and says nothing about

difficulty scaling. The intro says "three optional features" but there are four tasks.

Suggested fix: Rewrite the Task 2 text.

#: 14

Where: README.md:90-92 (README.md#L90-L92)

Problem: - [] doesn't render as a checkbox.

Suggested fix: Use - [].

#: 15

Where: repo root

Problem: game/__pycache__/ shows as untracked because there's no .gitignore.

Suggested fix: Add a .gitignore with __pycache__/ and *.pyc.

#: 16

Where: repo root

Problem: There's no requirements.txt. The code uses border_radius=, which needs

pygame 2.0 or newer and fails on 1.9.

Suggested fix: Add pygame>=2.0.

#: 17

Where: game/

Problem: There's no __init__.py. It still works as a namespace package, but adding an empty one is clearer and safer for tooling.

Suggested fix: Add an empty game/__init__.py.

5. Notes for the optional features

- Task 3 (speed tints): add a colour property in Anvil based on self.speed (e.g. grey below 5.5, orange below 6.5, red above) and use it in render.
- Task 4 (impact effects and screen shake): depends on fix #2. On ground impact, spawn particles stored in the engine and set a short shake timer. Render to an offscreen surface and blit it to the screen with a random offset while the timer is running.

When you're ready, tell me which of these you want me to implement. I'd start with #1 and #2, then #3 and #6.

> <ide_opened_file>The user opened the file

c:\PES\Third_year\Software_Engineering\63_anvil_dodge\game\game_engine.py in the IDE. This may or may not be related to the current

task.</ide_opened_file>

> yes start with 1 2 3 and 6

- I'm applying fixes #1, #2, #3 and #6 now.

- Bash(cd "c:/PES/Third_year/Software_Engineering/63_anvil_dodge" && python -

```
<<'EOF'
```

```
import re,io...)
```

```
└ Updated game\anvil.py (+29 -29)
```

```
1 -import random
```

```
2 -import pygame
```

```
3 -
```

```
4 -
```

```
5 -class Anvil:
```

```
6 -     def __init__(self, screen_width):
```

```
7 -         self.screen_width = screen_width
```

```
8 -         self.width = 40
```

```
9 -         self.height = 32
```

```
10 -         self.x = random.randint(20, screen_width - self.width -  
- 20)
```

```
11 -         self.y = -self.height
```

```
12 -         self.speed = random.uniform(4.5, 7.0)
```

```
13 -
```

```
14 -     def update(self):
```

```
15 -         self.y += self.speed
```

```
16 -
```

```
17 -     def is_off_screen(self, screen_height):
```

```
18 -         return self.y > screen_height + 10
```

```
19 -
```

```
20 -     @property
```

```

21 - def rect(self):
22 -     return pygame.Rect(int(self.x), int(self.y), self.width
    -, self.height)
23 -
24 - def render(self, surface):
25 -     top_rect = pygame.Rect(int(self.x) + 4, int(self.y), se
    -lf.width - 8, 14)
26 -     pygame.draw.rect(surface, (120, 120, 130), top_rect, bo
    -rder_radius=2)
27 -
28 -     base_rect = pygame.Rect(int(self.x), int(self.y) + 14,
    -self.width, 18)
29 -     pygame.draw.rect(surface, (80, 80, 90), base_rect, bord
    -er_radius=3)
1 +import random
2 +import pygame
3 +
4 +
5 +class Anvil:
6 + def __init__(self, screen_width):
7 +     self.screen_width = screen_width
8 +     self.width = 40
9 +     self.height = 32
10 +     self.x = random.randint(20, screen_width - self.width -
    + 20)
11 +     self.y = -self.height
... 19 more lines (ctrl+o to expand)

```

└ Updated game\game_engine.py (+96 -94)

```

1 -import pygame
2 -from game.player import Player
3 -from game.anvil import Anvil

```

```
4 -
5 -
6 -class GameEngine:
7 -    def __init__(self, width, height):
8 -        self.width = width
9 -        self.height = height
10 -        self.player = Player(width, height)
11 -        self anvils = []
12 -
13 -        self.spawn_delay = 700
14 -        self.last_spawn_time = pygame.time.get_ticks()
15 -
16 -        self.start_ticks = pygame.time.get_ticks()
17 -        self.survival_time = 0
18 -        self.game_state = "PLAYING"
19 -
20 -        self.font_big = pygame.font.SysFont(None, 52)
21 -        self.font_medium = pygame.font.SysFont(None, 34)
22 -        self.font_small = pygame.font.SysFont(None, 24)
23 -
24 -    def handle_event(self, event):
25 -        if self.game_state == "GAME_OVER":
26 -            if event.type == pygame.KEYDOWN and event.key == py
27 -                -game.K_r:
28 -                    self.reset()
29 -
30 -    def update(self):
31 -        if self.game_state != "PLAYING":
32 -            return
33 -        keys = pygame.key.get_pressed()
```

```
34 -     if keys[pygame.K_LEFT] or keys[pygame.K_a]:
35 -         self.player.move_left()
36 -     if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
37 -         self.player.move_right()
38 -
39 -     self.player.update()
40 -
```

... 151 more lines (ctrl+o to expand)

└ Updated game\player.py (+34 -34)

```
1 -import pygame
2 -
3 -
4 -class Player:
5 -     def __init__(self, screen_width, screen_height):
6 -         self.screen_width = screen_width
7 -         self.screen_height = screen_height
8 -         self.width = 44
9 -         self.height = 54
10 -        self.speed = 7
11 -
12 -        self.x = (screen_width // 2) - (self.width // 2)
13 -        self.y = screen_height - self.height - 20
14 -
15 -    def move_left(self):
16 -        self.x -= self.speed
17 -
18 -    def move_right(self):
19 -        self.x += self.speed
20 -
21 -    def update(self):
22 -        pass
```

```

23 -
24 - @property
25 - def rect(self):
26 -     return pygame.Rect(int(self.x), int(self.y), self.width
    -, self.height)
27 -
28 - def render(self, surface):
29 -     torso_rect = pygame.Rect(int(self.x) + 6, int(self.y) +
    - 18, 32, 36)
30 -     pygame.draw.rect(surface, (50, 130, 220), torso_rect, b
    -order_radius=4)
31 -
32 -     pygame.draw.circle(surface, (230, 200, 160), (int(self.
    -x) + 22, int(self.y) + 12), 12)
33 -     helmet_rect = pygame.Rect(int(self.x) + 8, int(self.y)
    -- 2, 28, 12)
34 -     pygame.draw.rect(surface, (180, 180, 190), helmet_rect,
    - border_radius=4)

```

```

1 +import pygame

```

```

2 +

```

```

3 +

```

```

4 +class Player:

```

```

5 +     def __init__(self, screen_width, screen_height):

```

```

6 +         self.screen_width = screen_width

```

```

... 28 more lines (ctrl+o to expand)

```

```

└ Updated main.py (+34 -32)

```

```

1 -import pygame

```

```

2 -from game.game_engine import GameEngine

```

```

3 -

```

```

4 -WIDTH, HEIGHT = 700, 500

```

```

5 -FPS = 60

```

```

6 -
7 -
8 -def main():
9 -    pygame.init()
10 -    screen = pygame.display.set_mode((WIDTH, HEIGHT))
11 -    pygame.display.set_caption("Anvil Dodge - Pygame Edition")
12 -    clock = pygame.time.Clock()
13 -
14 -    engine = GameEngine(WIDTH, HEIGHT)
15 -
16 -    running = True
17 -    while running:
18 -        for event in pygame.event.get():
19 -            if event.type == pygame.QUIT:
20 -                running = False
21 -            engine.handle_event(event)
22 -
23 -        engine.update()
24 -        engine.render(screen)
25 -
26 -        pygame.display.flip()
27 -        clock.tick(FPS)
28 -
29 -    pygame.quit()
30 -
31 -
32 -if __name__ == "__main__":
33 -    1 +import pygame
34 -    2 +from game.game_engine import GameEngine
35 -    3 +
36 -    4 +WIDTH, HEIGHT = 700, 500

```

5 +FPS = 60

6 +MAX_DT = 0.05 # cap frame time so a stalled window doesn't tel

+eport objects

7 +

8 +

... 27 more lines (ctrl+o to expand)

- The diff touched every line because my script saved the files with LF line endings, and the originals were CRLF. I'm changing them back.
- Bash(cd "c:/PES/Third_year/Software_Engineering/63_anvil_dodge" && for f in main.py game/anvil.py game/game_engine.py game/player.py; do sed -i 's/\$/\r/' "\$f"; done;...)
- └ Updated game\anvil.py (+30 -30)
 - 1 -import random
 - 2 -import pygame
 - 3 -
 - 4 -
 - 5 -class Anvil:
 - 6 - def __init__(self, screen_width):
 - 7 - self.screen_width = screen_width
 - 8 - self.width = 40
 - 9 - self.height = 32
 - 10 - self.x = random.randint(20, screen_width - self.width -
 - 20)
 - 11 - self.y = -self.height
 - 12 - self.speed = random.uniform(270, 420) # pixels per sec
 - ond
 - 13 -
 - 14 - def update(self, dt):
 - 15 - self.y += self.speed * dt

```

16 -
17 - def has_hit_ground(self, ground_y):
18 -     return self.y + self.height >= ground_y
19 -
20 - @property
21 - def rect(self):
22 -     return pygame.Rect(int(self.x), int(self.y), self.width
    -, self.height)
23 -
24 - def render(self, surface):
25 -     top_rect = pygame.Rect(int(self.x) + 4, int(self.y), se
    -lf.width - 8, 14)
26 -     pygame.draw.rect(surface, (120, 120, 130), top_rect, bo
    -rder_radius=2)
27 -
28 -     base_rect = pygame.Rect(int(self.x), int(self.y) + 14,
    -self.width, 18)
29 -     pygame.draw.rect(surface, (80, 80, 90), base_rect, bord
    -er_radius=3)
30 -     pygame.draw.rect(surface, (200, 200, 210), base_rect, w
    -idth=1, border_radius=3)
31 -
32 -
33 -
34 -
35 -
36 -
37 -
38 -
39 -
40 -
41 -
42 -
43 -
44 -
45 -
46 -
47 -
48 -
49 -
50 -
51 -
52 -
53 -
54 -
55 -
56 -
57 -
58 -
59 -
60 -
61 -
62 -
63 -
64 -
65 -
66 -
67 -
68 -
69 -
70 -
71 -
72 -
73 -
74 -
75 -
76 -
77 -
78 -
79 -
80 -
81 -
82 -
83 -
84 -
85 -
86 -
87 -
88 -
89 -
90 -
91 -
92 -
93 -
94 -
95 -
96 -
97 -
98 -
99 -
100 -
101 -
102 -
103 -
104 -
105 -
106 -
107 -
108 -
109 -
110 -
111 -
112 -
113 -
114 -
115 -
116 -
117 -
118 -
119 -
120 -
121 -
122 -
123 -
124 -
125 -
126 -
127 -
128 -
129 -
130 -
131 -
132 -
133 -
134 -
135 -
136 -
137 -
138 -
139 -
140 -
141 -
142 -
143 -
144 -
145 -
146 -
147 -
148 -
149 -
150 -
151 -
152 -
153 -
154 -
155 -
156 -
157 -
158 -
159 -
160 -
161 -
162 -
163 -
164 -
165 -
166 -
167 -
168 -
169 -
170 -
171 -
172 -
173 -
174 -
175 -
176 -
177 -
178 -
179 -
180 -
181 -
182 -
183 -
184 -
185 -
186 -
187 -
188 -
189 -
190 -
191 -
192 -
193 -
194 -
195 -
196 -
197 -
198 -
199 -
200 -
201 -
202 -
203 -
204 -
205 -
206 -
207 -
208 -
209 -
210 -
211 -
212 -
213 -
214 -
215 -
216 -
217 -
218 -
219 -
220 -
221 -
222 -
223 -
224 -
225 -
226 -
227 -
228 -
229 -
230 -
231 -
232 -
233 -
234 -
235 -
236 -
237 -
238 -
239 -
240 -
241 -
242 -
243 -
244 -
245 -
246 -
247 -
248 -
249 -
250 -
251 -
252 -
253 -
254 -
255 -
256 -
257 -
258 -
259 -
260 -
261 -
262 -
263 -
264 -
265 -
266 -
267 -
268 -
269 -
270 -
271 -
272 -
273 -
274 -
275 -
276 -
277 -
278 -
279 -
280 -
281 -
282 -
283 -
284 -
285 -
286 -
287 -
288 -
289 -
290 -
291 -
292 -
293 -
294 -
295 -
296 -
297 -
298 -
299 -
300 -
301 -
302 -
303 -
304 -
305 -
306 -
307 -
308 -
309 -
310 -
311 -
312 -
313 -
314 -
315 -
316 -
317 -
318 -
319 -
320 -
321 -
322 -
323 -
324 -
325 -
326 -
327 -
328 -
329 -
330 -
331 -
332 -
333 -
334 -
335 -
336 -
337 -
338 -
339 -
340 -
341 -
342 -
343 -
344 -
345 -
346 -
347 -
348 -
349 -
350 -
351 -
352 -
353 -
354 -
355 -
356 -
357 -
358 -
359 -
360 -
361 -
362 -
363 -
364 -
365 -
366 -
367 -
368 -
369 -
370 -
371 -
372 -
373 -
374 -
375 -
376 -
377 -
378 -
379 -
380 -
381 -
382 -
383 -
384 -
385 -
386 -
387 -
388 -
389 -
390 -
391 -
392 -
393 -
394 -
395 -
396 -
397 -
398 -
399 -
400 -
401 -
402 -
403 -
404 -
405 -
406 -
407 -
408 -
409 -
410 -
411 -
412 -
413 -
414 -
415 -
416 -
417 -
418 -
419 -
420 -
421 -
422 -
423 -
424 -
425 -
426 -
427 -
428 -
429 -
430 -
431 -
432 -
433 -
434 -
435 -
436 -
437 -
438 -
439 -
440 -
441 -
442 -
443 -
444 -
445 -
446 -
447 -
448 -
449 -
450 -
451 -
452 -
453 -
454 -
455 -
456 -
457 -
458 -
459 -
460 -
461 -
462 -
463 -
464 -
465 -
466 -
467 -
468 -
469 -
470 -
471 -
472 -
473 -
474 -
475 -
476 -
477 -
478 -
479 -
480 -
481 -
482 -
483 -
484 -
485 -
486 -
487 -
488 -
489 -
490 -
491 -
492 -
493 -
494 -
495 -
496 -
497 -
498 -
499 -
500 -
501 -
502 -
503 -
504 -
505 -
506 -
507 -
508 -
509 -
510 -
511 -
512 -
513 -
514 -
515 -
516 -
517 -
518 -
519 -
520 -
521 -
522 -
523 -
524 -
525 -
526 -
527 -
528 -
529 -
530 -
531 -
532 -
533 -
534 -
535 -
536 -
537 -
538 -
539 -
540 -
541 -
542 -
543 -
544 -
545 -
546 -
547 -
548 -
549 -
550 -
551 -
552 -
553 -
554 -
555 -
556 -
557 -
558 -
559 -
560 -
561 -
562 -
563 -
564 -
565 -
566 -
567 -
568 -
569 -
570 -
571 -
572 -
573 -
574 -
575 -
576 -
577 -
578 -
579 -
580 -
581 -
582 -
583 -
584 -
585 -
586 -
587 -
588 -
589 -
590 -
591 -
592 -
593 -
594 -
595 -
596 -
597 -
598 -
599 -
600 -
601 -
602 -
603 -
604 -
605 -
606 -
607 -
608 -
609 -
610 -
611 -
612 -
613 -
614 -
615 -
616 -
617 -
618 -
619 -
620 -
621 -
622 -
623 -
624 -
625 -
626 -
627 -
628 -
629 -
630 -
631 -
632 -
633 -
634 -
635 -
636 -
637 -
638 -
639 -
640 -
641 -
642 -
643 -
644 -
645 -
646 -
647 -
648 -
649 -
650 -
651 -
652 -
653 -
654 -
655 -
656 -
657 -
658 -
659 -
660 -
661 -
662 -
663 -
664 -
665 -
666 -
667 -
668 -
669 -
670 -
671 -
672 -
673 -
674 -
675 -
676 -
677 -
678 -
679 -
680 -
681 -
682 -
683 -
684 -
685 -
686 -
687 -
688 -
689 -
690 -
691 -
692 -
693 -
694 -
695 -
696 -
697 -
698 -
699 -
700 -
701 -
702 -
703 -
704 -
705 -
706 -
707 -
708 -
709 -
710 -
711 -
712 -
713 -
714 -
715 -
716 -
717 -
718 -
719 -
720 -
721 -
722 -
723 -
724 -
725 -
726 -
727 -
728 -
729 -
730 -
731 -
732 -
733 -
734 -
735 -
736 -
737 -
738 -
739 -
740 -
741 -
742 -
743 -
744 -
745 -
746 -
747 -
748 -
749 -
750 -
751 -
752 -
753 -
754 -
755 -
756 -
757 -
758 -
759 -
760 -
761 -
762 -
763 -
764 -
765 -
766 -
767 -
768 -
769 -
770 -
771 -
772 -
773 -
774 -
775 -
776 -
777 -
778 -
779 -
780 -
781 -
782 -
783 -
784 -
785 -
786 -
787 -
788 -
789 -
790 -
791 -
792 -
793 -
794 -
795 -
796 -
797 -
798 -
799 -
800 -
801 -
802 -
803 -
804 -
805 -
806 -
807 -
808 -
809 -
810 -
811 -
812 -
813 -
814 -
815 -
816 -
817 -
818 -
819 -
820 -
821 -
822 -
823 -
824 -
825 -
826 -
827 -
828 -
829 -
830 -
831 -
832 -
833 -
834 -
835 -
836 -
837 -
838 -
839 -
840 -
841 -
842 -
843 -
844 -
845 -
846 -
847 -
848 -
849 -
850 -
851 -
852 -
853 -
854 -
855 -
856 -
857 -
858 -
859 -
860 -
861 -
862 -
863 -
864 -
865 -
866 -
867 -
868 -
869 -
870 -
871 -
872 -
873 -
874 -
875 -
876 -
877 -
878 -
879 -
880 -
881 -
882 -
883 -
884 -
885 -
886 -
887 -
888 -
889 -
890 -
891 -
892 -
893 -
894 -
895 -
896 -
897 -
898 -
899 -
900 -
901 -
902 -
903 -
904 -
905 -
906 -
907 -
908 -
909 -
910 -
911 -
912 -
913 -
914 -
915 -
916 -
917 -
918 -
919 -
920 -
921 -
922 -
923 -
924 -
925 -
926 -
927 -
928 -
929 -
930 -
931 -
932 -
933 -
934 -
935 -
936 -
937 -
938 -
939 -
940 -
941 -
942 -
943 -
944 -
945 -
946 -
947 -
948 -
949 -
950 -
951 -
952 -
953 -
954 -
955 -
956 -
957 -
958 -
959 -
960 -
961 -
962 -
963 -
964 -
965 -
966 -
967 -
968 -
969 -
970 -
971 -
972 -
973 -
974 -
975 -
976 -
977 -
978 -
979 -
980 -
981 -
982 -
983 -
984 -
985 -
986 -
987 -
988 -
989 -
990 -
991 -
992 -
993 -
994 -
995 -
996 -
997 -
998 -
999 -

```

+ 20)

... 20 more lines (ctrl+o to expand)

└ Updated game\game_engine.py (+97 -97)

1 -import pygame

2 -from game.player import Player

3 -from game.anvil import Anvil

4 -

5 -

6 -class GameEngine:

7 - def __init__(self, width, height):

8 - self.width = width

9 - self.height = height

10 - self.ground_y = height - 20

11 - self.player = Player(width, height)

12 - self.anvils = []

13 -

14 - self.spawn_delay = 0.7 # seconds

15 - self.spawn_timer = 0.0

16 -

17 - self.elapsed_time = 0.0

18 - self.survival_time = 0

19 - self.game_state = "PLAYING"

20 -

21 - self.font_big = pygame.font.SysFont(None, 52)

22 - self.font_medium = pygame.font.SysFont(None, 34)

23 - self.font_small = pygame.font.SysFont(None, 24)

24 -

25 - def handle_event(self, event):

26 - if self.game_state == "GAME_OVER":

27 - if event.type == pygame.KEYDOWN and event.key == py

-game.K_r:

```

28 -         self.reset()
29 -
30 - def update(self, dt):
31 -     if self.game_state != "PLAYING":
32 -         return
33 -
34 -     keys = pygame.key.get_pressed()
35 -     if keys[pygame.K_LEFT] or keys[pygame.K_a]:
36 -         self.player.move_left(dt)
37 -     if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
38 -         self.player.move_right(dt)
39 -
40 -     self.player.update()

```

... 154 more lines (ctrl+o to expand)

└ Updated game\player.py (+34 -34)

```

1 -import pygame
2 -
3 -
4 -class Player:
5 -    def __init__(self, screen_width, screen_height):
6 -        self.screen_width = screen_width
7 -        self.screen_height = screen_height
8 -        self.width = 44
9 -        self.height = 54
10 -        self.speed = 420 # pixels per second
11 -
12 -        self.x = (screen_width // 2) - (self.width // 2)
13 -        self.y = screen_height - self.height - 20
14 -
15 -    def move_left(self, dt):
16 -        self.x -= self.speed * dt

```

```

17 -
18 - def move_right(self, dt):
19 -     self.x += self.speed * dt
20 -
21 - def update(self):
22 -     self.x = max(0, min(self.x, self.screen_width - self.wi
        -dth))
23 -
24 - @property
25 - def rect(self):
26 -     return pygame.Rect(int(self.x), int(self.y), self.width
        -, self.height)
27 -
28 - def render(self, surface):
29 -     torso_rect = pygame.Rect(int(self.x) + 6, int(self.y) +
        - 18, 32, 36)
30 -     pygame.draw.rect(surface, (50, 130, 220), torso_rect, b
        -order_radius=4)
31 -
32 -     pygame.draw.circle(surface, (230, 200, 160), (int(self.
        -x) + 22, int(self.y) + 12), 12)
33 -     helmet_rect = pygame.Rect(int(self.x) + 8, int(self.y)
        -- 2, 28, 12)
34 -     pygame.draw.rect(surface, (180, 180, 190), helmet_rect,
        - border_radius=4)
1 +import pygame
2 +
3 +
4 +class Player:
5 +     def __init__(self, screen_width, screen_height):
6 +         self.screen_width = screen_width

```

... 28 more lines (ctrl+o to expand)

└ Updated main.py (+35 -35)

```
1 -import pygame
2 -from game.game_engine import GameEngine
3 -
4 -WIDTH, HEIGHT = 700, 500
5 -FPS = 60
6 -MAX_DT = 0.05 # cap frame time so a stalled window doesn't tel
   -eport objects
7 -
8 -
9 -def main():
10 -    pygame.init()
11 -    screen = pygame.display.set_mode((WIDTH, HEIGHT))
12 -    pygame.display.set_caption("Anvil Dodge - Pygame Edition")
13 -    clock = pygame.time.Clock()
14 -
15 -    engine = GameEngine(WIDTH, HEIGHT)
16 -
17 -    running = True
18 -    while running:
19 -        dt = min(clock.tick(FPS) / 1000, MAX_DT)
20 -
21 -        for event in pygame.event.get():
22 -            if event.type == pygame.QUIT:
23 -                running = False
24 -            engine.handle_event(event)
25 -
26 -        engine.update(dt)
27 -        engine.render(screen)
28 -
```

```

29 -     pygame.display.flip()
30 -
31 -     pygame.quit()
32 -
33 -
34 -if __name__ == "__main__":
35 -    main()
36 -
37 +import pygame
38 +from game.game_engine import GameEngine
39 +
40 +
41 +WIDTH, HEIGHT = 700, 500
42 +
43 +FPS = 60
44 +
45 ... 30 more lines (ctrl+o to expand)

```

- The last line of three files picked up a stray `\r`. I'll remove it, then run a headless check.

Ran 1 shell command (ctrl+o to expand)

- Bash(`cd "c:/PES/Third_year/Software_Engineering/63_anvil_dodge" &&`
`S="/c/Users/Krish/AppData/Local/Temp/claude/c--PES-Third-year-Software-Eng`
`ineering-63-anvil-dodge/..."`
`_ 3 files changed (binary, mode only or too large to show)`)

- I've made fixes #1, #2, #3 and #6, and a headless pygame test of each one passed.

- #1, player boundary: `Player.update()` now keeps the player between 0 and `screen_width - width` (`player.py:21-22 (game/player.py#L21-L22)`). The test held each direction for 500 frames and the player stopped exactly at `x=0` and `x=656`.

- #2, ground removal: anvils are now removed when they touch the ground line, not after falling off the bottom of the screen. `is_off_screen()` became `has_hit_ground(ground_y)` (anvil.py:17-18 (game/anvil.py#L17-L18)). The engine now stores `self.ground_y` and uses it for both removal and drawing. In the test, no anvil was drawn below 474.6 px, and the ground is at 480. This is also the spot where the Task 4 impact effects will go.
- #3, collision break: once `GAME_OVER` is set, the anvil loop stops, so the other anvils don't move during that frame (game_engine.py:58 (game/game_engine.py#L58)).
- #6, frame-rate independence:
 - main.py now works out `dt` (the time since the last frame) at the top of each loop, capped at 0.05 s. This stops objects jumping forward when the window is dragged on Windows.
 - Player and anvil speeds are now in pixels per second: 420 for the player and 270–420 for anvils, the same as before at 60 FPS.
 - Spawning and survival time now add up `dt` instead of reading `get_ticks()`, so they pause whenever the game loop pauses.
 - `reset()` clears the new timers.

One small thing: survival time is now a sum of frame times, so it can reach a new second one frame (about 16 ms) late. For example, 120 frames at 1/60 s added up to 1.999..., which still shows as 1 s. You won't notice it in play.

When my script edited the files it also changed their line endings. I changed them back, so the diff only shows real changes. I haven't committed anything. The README tasks still left are #2 (difficulty scaling), #3 (speed tints) and #4 (impact effects), plus the smaller cleanups from the list.